本页文件：S1 run.sh；S2 server.py

```
01 S1:0001  #!/usr/bin/env bash
02 S1:0002  set -euo pipefail
03 S1:0003  cd "$(dirname "$0")"
04 S1:0004  if [[ -n "${CHEMDB_PYTHON:-}" ]]; then
05 S1:0005    interpreter="$CHEMDB_PYTHON"
06 S1:0006  elif [[ -x /home/mytu/Software/miniforge3/envs/drug_info/bin/python ]]; then
07 S1:0007    interpreter=/home/mytu/Software/miniforge3/envs/drug_info/bin/python
08 S1:0008  else
09 S1:0009    interpreter=python3
10 S1:0010  fi
11 S1:0011  exec "$interpreter" server.py "$@"
12 S2:0001  """Loopback-only web application. Start with python server.py."""
13 S2:0002  import argparse
14 S2:0003  import csv
15 S2:0004  import io
16 S2:0005  import json
17 S2:0006  import mimetypes
18 S2:0007  from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
19 S2:0008  from pathlib import Path
20 S2:0009  from urllib.parse import parse_qs, urlsplit
21 S2:0011  from chemdb import Store, structure
22 S2:0013  ROOT = Path(__file__).resolve().parent
23 S2:0016  def safe_cell(value):
24 S2:0017      if isinstance(value, (int, float)):
25 S2:0018          return value
26 S2:0019      s = '' if value is None else str(value)
27 S2:0020      return "'" + s if s.lstrip().startswith(('=', '+', '-', '@', '\t', '\r')) else s
28 S2:0023  class Handler(BaseHTTPRequestHandler):
29 S2:0024      def send(self, body, content_type='application/json; charset=utf-8', status=200, filename=No
30 S2:0024+ ne):
31 S2:0025          if not isinstance(body, (str, bytes)):
32 S2:0026              body = json.dumps(body, ensure_ascii=False, allow_nan=False)
33 S2:0027          if isinstance(body, str):
34 S2:0028              body = body.encode('utf-8')
35 S2:0029          self.send_response(status)
36 S2:0030          self.send_header('Content-Type', content_type)
37 S2:0031          self.send_header('Content-Length', str(len(body)))
38 S2:0032          self.send_header('X-Content-Type-Options', 'nosniff')
39 S2:0033          self.send_header('Cache-Control', 'no-store')
40 S2:0034          if filename:
41 S2:0035              self.send_header('Content-Disposition', f'attachment; filename="{filename}"')
42 S2:0036          self.end_headers()
43 S2:0037          self.wfile.write(body)
44 S2:0039      def do_GET(self):
45 S2:0040          try:
46 S2:0041              self.route_get()
47 S2:0042          except (ValueError, TypeError, KeyError) as e:
48 S2:0043              self.send({'error': str(e)}, status=400)
49 S2:0044          except Exception as e:
50 S2:0045              self.log_error('%s', e)
51 S2:0046              self.send({'error': '服务处理失败，请查看运行终端'}, status=500)
52 S2:0048      def route_get(self):
```

本页文件：S2 server.py

```
S2:0049           url = urlsplit(self.path)
S2:0050           p = {k: v[0] for k, v in parse_qs(url.query).items()}
S2:0051           store = self.server.store
S2:0052           path = url.path
S2:0053           if path == '/api/overview' or path == '/api/quality':
S2:0054               self.send(store.quality())
S2:0055           elif path == '/api/search':
S2:0056               self.send(store.search(p))
S2:0057           elif path == '/api/detail':
S2:0058               self.send(store.detail(p['id']))
S2:0059           elif path == '/api/structure':
S2:0060               self.send(structure(store.detail(p['id'])['smiles']), 'image/svg+xml')
S2:0061           elif path == '/api/similar':
S2:0062               self.send(store.similar(p.get('smiles'), p.get('threshold', .4)))
S2:0063           elif path == '/api/screen':
S2:0064               self.send(store.screen(p))
S2:0065           elif path == '/api/runs':
S2:0066               self.send(store.runs())
S2:0067           elif path == '/api/run':
S2:0068               self.send(store.run(int(p['id'])))
S2:0069           elif path == '/api/export':
S2:0070               self.export(p)
S2:0071           else:
S2:0072               relative = 'index.html' if path == '/' else path.lstrip('/')
S2:0073               file = (ROOT / 'web' / relative).resolve()
S2:0074               if not file.is_relative_to(ROOT / 'web') or not file.is_file():
S2:0075                   self.send({'error': '页面不存在'}, status=404)
S2:0076                   return
S2:0077               mime = mimetypes.guess_type(file.name)[0] or 'application/octet-stream'
S2:0078               self.send(file.read_bytes(), mime + ('; charset=utf-8' if mime.startswith('text/') e
S2:0078+ lse ''))
S2:0080       def export(self, p):
S2:0081           store = self.server.store
S2:0082           if p.get('kind') == 'run':
S2:0083               data = store.run(int(p['id']))
S2:0084               self.send(data, filename=f'screening-{data["id"]}.json')
S2:0085               return
S2:0086           if p.get('kind') == 'screen':
S2:0087               result = store.screen(p)
S2:0088               ids = result['ids']
S2:0089           else:
S2:0090               p = dict(p, size=100, page=1)
S2:0091               first = store.search(p)
S2:0092               ids = [r['id'] for r in first['items']]
S2:0093               for page in range(2, (first['total'] + 99) // 100 + 1):
S2:0094                   ids.extend(r['id'] for r in store.search(dict(p, page=page))['items'])
S2:0095           output = io.StringIO()
S2:0096           fields = ['id', 'source', 'source_row', 'name', 'smiles', 'canonical', 'mw', 'logp', 'hb
S2:0096+ d', 'hba', 'psa', 'label']
S2:0097           writer = csv.writer(output)
S2:0098           writer.writerow(fields)
S2:0099           with store.connect() as db:
```

本页文件：S2 server.py；S3 chemdb.py

```
S2:0100                for identifier in ids:
S2:0101                    row = db.execute('SELECT * FROM records WHERE id=?', (identifier,)).fetchone()
S2:0102                    writer.writerow(safe_cell(row[k]) for k in fields)
S2:0103            self.send('\ufeff' + output.getvalue(), 'text/csv; charset=utf-8', filename='chemical-re
S2:0103+ sults.csv')
S2:0105        def do_POST(self):
S2:0106            try:
S2:0107                host = self.headers.get('Host', '')
S2:0108                origin = self.headers.get('Origin')
S2:0109                if origin and origin != 'http://' + host:
S2:0110                    raise ValueError('不接受跨站写入')
S2:0111                if urlsplit(self.path).path != '/api/runs':
S2:0112                    self.send({'error': '接口不存在'}, status=404)
S2:0113                    return
S2:0114                length = int(self.headers.get('Content-Length', 0))
S2:0115                if not 0 < length <= 2000000:
S2:0116                    raise ValueError('请求体为空或过大')
S2:0117                if not self.headers.get('Content-Type', '').startswith('application/json'):
S2:0118                    raise ValueError('请求格式须为 JSON')
S2:0119                body = json.loads(self.rfile.read(length))
S2:0120                if not isinstance(body, dict):
S2:0121                    raise ValueError('请求体须为 JSON 对象')
S2:0122                self.send(self.server.store.save_run(body['name'], body['params'], body['ids']), sta
S2:0122+ tus=201)
S2:0123            except (ValueError, TypeError, KeyError) as e:
S2:0124                self.send({'error': str(e)}, status=400)
S2:0127    def main():
S2:0128        parser = argparse.ArgumentParser()
S2:0129        parser.add_argument('--port', type=int, default=8765)
S2:0130        args = parser.parse_args()
S2:0131        store = Store(ROOT)
S2:0132        server = ThreadingHTTPServer(('127.0.0.1', args.port), Handler)
S2:0133        server.store = store
S2:0134        print(f'化合物数据与可解释筛选系统 V1.0 — http://127.0.0.1:{args.port}', flush=True)
S2:0135        try:
S2:0136            server.serve_forever()
S2:0137        except KeyboardInterrupt:
S2:0138            server.server_close()
S2:0140    if __name__ == '__main__':
S2:0141        main()
S3:0001    """Local chemical data indexing, reproducible screening, and structure analysis."""
S3:0002    import csv
S3:0003    import hashlib
S3:0004    import json
S3:0005    import math
S3:0006    import sqlite3
S3:0007    from datetime import datetime, timezone
S3:0008    from pathlib import Path
S3:0010    from openpyxl import load_workbook
S3:0011    from rdkit import Chem, DataStructs, RDLogger, rdBase
S3:0012    from rdkit.Chem import Descriptors, Crippen, Lipinski, rdMolDescriptors, rdFingerprintGenerator
S3:0013    from rdkit.Chem.Draw import rdMolDraw2D
```

本页文件：S3 chemdb.py

```
S3:0015  RDLogger.DisableLog('rdApp.*')
S3:0016  GENERATOR = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=2048)
S3:0017  SOURCES = {'compound': 'compound_info.csv', 'bbb': 'Dataset/BBB_10648.csv', 'drugbank': 'DrugBan
S3:0017+ kAll.xlsx'}
S3:0018  PROPERTIES = ('mw', 'logp', 'hbd', 'hba', 'psa')
S3:0021  def molecule(smiles):
S3:0022      if not isinstance(smiles, str) or not smiles.strip() or len(smiles) > 10000:
S3:0023          raise ValueError('请输入有效 SMILES，长度不超过 10000 字符')
S3:0024      mol = Chem.MolFromSmiles(smiles.strip())
S3:0025      if mol is None or mol.GetNumAtoms() == 0 or mol.GetNumAtoms() > 1000:
S3:0026          raise ValueError('SMILES 无法解析或分子超过 1000 个原子，请检查环闭合、括号与元素符号')
S3:0027      return mol
S3:0030  def analyze(smiles):
S3:0031      mol = molecule(smiles)
S3:0032      return {'canonical': Chem.MolToSmiles(mol), 'mw': round(Descriptors.MolWt(mol), 3),
S3:0033              'logp': round(Crippen.MolLogP(mol), 3), 'hbd': Lipinski.NumHDonors(mol),
S3:0034              'hba': Lipinski.NumHAcceptors(mol), 'psa': round(rdMolDescriptors.CalcTPSA(mol), 3)}
S3:0037  def structure(smiles):
S3:0038      mol = molecule(smiles)
S3:0039      drawer = rdMolDraw2D.MolDraw2DSVG(600, 340)
S3:0040      drawer.drawOptions().padding = .12
S3:0041      drawer.DrawMolecule(mol)
S3:0042      drawer.FinishDrawing()
S3:0043      return drawer.GetDrawingText()
S3:0046  def number(value, name, low, high):
S3:0047      try:
S3:0048          n = float(value)
S3:0049      except (ValueError, TypeError):
S3:0050          raise ValueError(f'{name} 必须是数值') from None
S3:0051      if not math.isfinite(n) or n < low or n > high:
S3:0052          raise ValueError(f'{name} 必须在 {low} 至 {high} 之间')
S3:0053      return n
S3:0056  class Store:
S3:0057      def __init__(self, root, db_path=None):
S3:0058          self.root = Path(root)
S3:0059          self.path = Path(db_path or self.root / 'runtime/chemical.sqlite')
S3:0060          self.path.parent.mkdir(parents=True, exist_ok=True)
S3:0061          self._bootstrap()
S3:0062          with self.connect() as db:
S3:0063              rows = db.execute('SELECT id,canonical FROM records WHERE canonical IS NOT NULL').fe
S3:0063+ tchall()
S3:0064          # Each unique structure is fingerprinted once, but all source rows are retained.
S3:0065          self.fp_cache = {}
S3:0066          self.fp_rows = []
S3:0067          for row in rows:
S3:0068              smi = row['canonical']
S3:0069              if smi not in self.fp_cache:
S3:0070                  self.fp_cache[smi] = GENERATOR.GetFingerprint(Chem.MolFromSmiles(smi))
S3:0071              self.fp_rows.append((row['id'], self.fp_cache[smi]))
S3:0073      def connect(self):
S3:0074          db = sqlite3.connect(self.path, timeout=30)
S3:0075          db.row_factory = sqlite3.Row
```

本页文件：S3 chemdb.py

```
01 S3:0076            return db
02 S3:0078        def _bootstrap(self):
03 S3:0079            with self.connect() as db:
04 S3:0080                db.executescript('''
05 S3:0081                CREATE TABLE IF NOT EXISTS records (
06 S3:0082                  id TEXT PRIMARY KEY, source TEXT, source_row INTEGER, name TEXT,
07 S3:0083                  smiles TEXT, canonical TEXT, mw REAL, logp REAL, hbd INTEGER, hba INTEGER,
08 S3:0084                  psa REAL, label TEXT, payload TEXT, issue TEXT);
09 S3:0085                CREATE INDEX IF NOT EXISTS source_idx ON records(source);
10 S3:0086                CREATE INDEX IF NOT EXISTS canonical_idx ON records(canonical);
11 S3:0087                CREATE TABLE IF NOT EXISTS sources (source TEXT PRIMARY KEY, path TEXT, sha256 TEXT,
12 S3:0088                  rows INTEGER, invalid INTEGER, missing INTEGER, duplicates INTEGER, imported TEXT)
13 S3:0088+  ;
14 S3:0089                CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY AUTOINCREMENT,
15 S3:0090                  name TEXT, created TEXT, params TEXT, ids TEXT, provenance TEXT);
16 S3:0091                ''')
17 S3:0092                for source, relative in SOURCES.items():
18 S3:0093                    path = self.root / relative
19 S3:0094                    digest = hashlib.sha256(path.read_bytes()).hexdigest()
20 S3:0095                    old = db.execute('SELECT sha256 FROM sources WHERE source=?', (source,)).fetchon
21 S3:0095+  e()
22 S3:0096                    if old and old['sha256'] == digest:
23 S3:0097                        continue
24 S3:0098                    print(f'Indexing {relative}', flush=True)
25 S3:0099                    db.execute('DELETE FROM records WHERE source=?', (source,))
26 S3:0100                    seen = set()
27 S3:0101                    invalid = missing = duplicates = count = 0
28 S3:0102                    for count, row in enumerate(self._rows(path), start=1):
29 S3:0103                        smi = row.get('SMILES') or None
30 S3:0104                        name = row.get('ID') or row.get('DrugBank-ID') or f'BBB-{count:05d}'
31 S3:0105                        identifier = f'{source}:{name}'
32 S3:0106                        props = dict.fromkeys(('canonical', *PROPERTIES))
33 S3:0107                        issue = ''
34 S3:0108                        if smi:
35 S3:0109                            try:
36 S3:0110                                props = analyze(smi)
37 S3:0111                            except ValueError:
38 S3:0112                                invalid += 1
39 S3:0113                                issue = '无法解析结构'
40 S3:0114                        elif source != 'drugbank':
41 S3:0115                            missing += 1
42 S3:0116                            issue = '缺少结构'
43 S3:0117                        key = props['canonical'] or name
44 S3:0118                        if key in seen:
45 S3:0119                            duplicates += 1
46 S3:0120                        seen.add(key)
47 S3:0121                        payload = {k: v for k, v in row.items() if k and k != 'Mol'}
48 S3:0122                        label = row.get('if_active', row.get('BBB', ''))
49 S3:0123                        values = (identifier, source, count + 1, name, smi, props['canonical'],
50 S3:0124                                  *(props[p] for p in PROPERTIES), label,
51 S3:0125                                  json.dumps(payload, ensure_ascii=False), issue)
52 S3:0126                        db.execute('INSERT INTO records VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?)', value
```

左栏为页内行号及原文件行号；“+”表示同一源码行的续排。源文件全部纳入，空白行不显示。

本页 52 排版行；源码量按原文件统计，折行不增加源程序量。

本页文件：S3 chemdb.py

```
01 S3:0126+  s)
02 S3:0127                   db.execute('INSERT OR REPLACE INTO sources VALUES (?,?,?,?,?,?,?,?)',
03 S3:0128                              (source, relative, digest, count, invalid, missing, duplicates,
04 S3:0129                               datetime.now(timezone.utc).isoformat()))
05 S3:0131       @staticmethod
06 S3:0132       def _rows(path):
07 S3:0133           if path.suffix == '.csv':
08 S3:0134               with path.open(encoding='utf-8-sig', newline='') as handle:
09 S3:0135                   yield from csv.DictReader(handle)
10 S3:0136           else:
11 S3:0137               book = load_workbook(path, read_only=True, data_only=True)
12 S3:0138               rows = book.active.iter_rows(values_only=True)
13 S3:0139               headers = next(rows)
14 S3:0140               for values in rows:
15 S3:0141                   yield {key: '' if value is None else str(value) for key, value in zip(headers, v
16 S3:0141+  alues)}
17 S3:0142               book.close()
18 S3:0144       @staticmethod
19 S3:0145       def unpack(row):
20 S3:0146           result = dict(row)
21 S3:0147           if 'payload' in result:
22 S3:0148               result['payload'] = json.loads(result['payload'])
23 S3:0149           return result
24 S3:0151       def search(self, params):
25 S3:0152           page = int(number(params.get('page', 1), '页码', 1, 100000))
26 S3:0153           size = int(number(params.get('size', 12), '每页条数', 1, 100))
27 S3:0154           conditions, args = [], []
28 S3:0155           source = params.get('source', '')
29 S3:0156           if source:
30 S3:0157               if source not in SOURCES:
31 S3:0158                   raise ValueError('未知数据源')
32 S3:0159               conditions.append('source=?')
33 S3:0160               args.append(source)
34 S3:0161           q = str(params.get('q', '')).strip()
35 S3:0162           if len(q) > 1000:
36 S3:0163               raise ValueError('搜索词不能超过 1000 字符')
37 S3:0164           if q:
38 S3:0165               conditions.append('(instr(lower(name),lower(?))>0 OR instr(smiles,?)>0 OR instr(lowe
39 S3:0165+  r(payload),lower(?))>0)')
40 S3:0166               args.extend([q, q, q])
41 S3:0167           label = params.get('label', '')
42 S3:0168           if label:
43 S3:0169               conditions.append('label=?')
44 S3:0170               args.append(label)
45 S3:0171           where = ' WHERE ' + ' AND '.join(conditions) if conditions else ''
46 S3:0172           with self.connect() as db:
47 S3:0173               total = db.execute('SELECT count(*) FROM records' + where, args).fetchone()[0]
48 S3:0174               items = db.execute('SELECT id,source,name,smiles,mw,logp,hbd,hba,psa,label,issue FRO
49 S3:0174+  M records' + where +
50 S3:0175                                  ' ORDER BY source,id LIMIT ? OFFSET ?', [*args, size, (page - 1)
51 S3:0175+  * size]).fetchall()
52 S3:0176           return {'items': [dict(r) for r in items], 'total': total, 'page': page, 'size': size}
```

左栏为页内行号及原文件行号；“+”表示同一源码行的续排。源文件全部纳入，空白行不显示。

本页 52 排版行；源码量按原文件统计，折行不增加源程序量。

本页文件：S3 chemdb.py

```
S3:0178       def detail(self, identifier):
S3:0179           with self.connect() as db:
S3:0180               row = db.execute('SELECT * FROM records WHERE id=?', (identifier,)).fetchone()
S3:0181               if row is None:
S3:0182                   raise ValueError('记录不存在')
S3:0183               result = self.unpack(row)
S3:0184               result['related'] = [dict(r) for r in db.execute(
S3:0185                   'SELECT id,source,name,label FROM records WHERE canonical=? AND id!=? LIMIT 20',
S3:0186                   (result['canonical'], identifier))] if result['canonical'] else []
S3:0187               result['related_total'] = db.execute('SELECT count(*) FROM records WHERE canonical=?
S3:0187+  AND id!=?',
S3:0188                   (result['canonical'], identifier)).fetchone()[0] if result['canonical'] else 0
S3:0189               return result
S3:0191       def similar(self, smiles, threshold=.4):
S3:0192           threshold = number(threshold, '相似度阈值', 0, 1)
S3:0193           mol = molecule(smiles)
S3:0194           fp = GENERATOR.GetFingerprint(mol)
S3:0195           scores = DataStructs.BulkTanimotoSimilarity(fp, [pair[1] for pair in self.fp_rows])
S3:0196           matches = sorted(((score, self.fp_rows[i][0]) for i, score in enumerate(scores) if score
S3:0196+  >= threshold),
S3:0197                            key=lambda x: (-x[0], x[1]))
S3:0198           items = []
S3:0199           for score, identifier in matches[:60]:
S3:0200               record = self.detail(identifier)
S3:0201               record['similarity'] = round(score, 6)
S3:0202               items.append(record)
S3:0203           return {'items': items, 'total': len(matches), 'query': analyze(smiles), 'threshold': th
S3:0203+ reshold,
S3:0204                   'algorithm': 'Morgan radius=2 / 2048 bits / Tanimoto', 'limit': 60}
S3:0206       def screen(self, params):
S3:0207           if not isinstance(params, dict):
S3:0208               raise ValueError('筛选条件须为对象')
S3:0209           defaults = {'mw_max': 500, 'logp_max': 5, 'hbd_max': 5, 'hba_max': 10, 'psa_max': 140}
S3:0210           clean = {key: number(params.get(key, default), key, -20 if key == 'logp_max' else 0,
S3:0211                                20 if key == 'logp_max' else 10000) for key, default in defaults.i
S3:0211+ tems()}
S3:0212           source = params.get('source', 'compound')
S3:0213           if source not in ('compound', 'bbb', 'all'):
S3:0214               raise ValueError('筛选数据源必须有分子结构')
S3:0215           clean['source'] = source
S3:0216           conditions = ['canonical IS NOT NULL'] + [f'{p}<=?' for p in PROPERTIES]
S3:0217           args = [clean[p + '_max'] for p in PROPERTIES]
S3:0218           if source != 'all':
S3:0219               conditions.append('source=?')
S3:0220               args.append(source)
S3:0221           with self.connect() as db:
S3:0222               rows = db.execute('SELECT id,source,name,smiles,mw,logp,hbd,hba,psa,label FROM recor
S3:0222+ ds WHERE ' +
S3:0223                                 ' AND '.join(conditions) + ' ORDER BY mw,id', args).fetchall()
S3:0224           items = []
S3:0225           for row in rows[:100]:
S3:0226               r = dict(row)
```

本页文件：S3 chemdb.py；S4 web/index.html

```
S3:0227              r['reasons'] = [f'{p.upper()} {r[p]} ≤ {clean[p + "_max"]:g}' for p in PROPERTIES]
S3:0228              items.append(r)
S3:0229          return {'items': items, 'ids': [r['id'] for r in rows], 'total': len(rows), 'params': cl
S3:0229+ ean, 'limit': 100}
S3:0231      def quality(self):
S3:0232          with self.connect() as db:
S3:0233              sources = [dict(r) for r in db.execute('SELECT * FROM sources ORDER BY source')]
S3:0234              labels = [dict(r) for r in db.execute('SELECT source,label,count(*) AS count FROM re
S3:0234+ cords GROUP BY source,label')]
S3:0235              metrics = dict(db.execute('SELECT count(canonical) AS structures,count(DISTINCT cano
S3:0235+ nical) AS unique_structures FROM records').fetchone())
S3:0236              drug_missing = [dict(r) for r in db.execute("SELECT json_extract(payload,'$.State')
S3:0236+ AS state,count(*) AS count FROM records WHERE source='drugbank' GROUP BY state")]
S3:0237          return {'sources': sources, 'labels': labels, **metrics, 'drug_states': drug_missing,
S3:0238                  'rdkit': rdBase.rdkitVersion,
S3:0239                  'quality_note': '重复按各源内规范化 SMILES（无结构时按 ID）统计，保留全部来源行；DrugBank
S3:0239+  设计上无 SMILES。'}
S3:0241      def save_run(self, name, params, ids):
S3:0242          name = str(name).strip()
S3:0243          if not name or len(name) > 100:
S3:0244              raise ValueError('记录名称须为 1 至 100 字符')
S3:0245          # Recompute on the server: clients cannot forge an archived result list.
S3:0246          result = self.screen(params)
S3:0247          if ids != result['ids']:
S3:0248              raise ValueError('结果与条件不一致，请重新执行筛选')
S3:0249          provenance = {'sources': self.quality()['sources'], 'rdkit': rdBase.rdkitVersion,
S3:0250                        'algorithm': 'RDKit properties; inclusive upper bounds',
S3:0251                        'snapshot': result['items'], 'snapshot_limit': 100}
S3:0252          with self.connect() as db:
S3:0253              cur = db.execute('INSERT INTO runs(name,created,params,ids,provenance) VALUES (?,?,?
S3:0253+ ,?,?)',
S3:0254                               (name, datetime.now(timezone.utc).isoformat(), json.dumps(result['p
S3:0254+ arams']),
S3:0255                                json.dumps(ids), json.dumps(provenance, ensure_ascii=False)))
S3:0256              return {'id': cur.lastrowid}
S3:0258      def runs(self):
S3:0259          with self.connect() as db:
S3:0260              return [dict(r) for r in db.execute('SELECT id,name,created,json_array_length(ids) A
S3:0260+ S count FROM runs ORDER BY id DESC')]
S3:0262      def run(self, identifier):
S3:0263          with self.connect() as db:
S3:0264              row = db.execute('SELECT * FROM runs WHERE id=?', (identifier,)).fetchone()
S3:0265          if row is None:
S3:0266              raise ValueError('筛选记录不存在')
S3:0267          result = dict(row)
S3:0268          for field in ('params', 'ids', 'provenance'):
S3:0269              result[field] = json.loads(result[field])
S3:0270          return result
S4:0001  <!doctype html>
S4:0002  <html lang="zh-CN">
S4:0003  <head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1"><
```

本页文件：S4 web/index.html；S5 web/style.css

```
01 S4:0003+ title>化合物数据与可解释筛选系统 V1.0</title><link rel="stylesheet" href="/style.css"><script defer s
02 S4:0003+ rc="/app.js"></script></head>
03 S4:0004  <body>
04 S4:0005  <aside><a class="brand" href="#overview"><span class="brand-icon">⌬</span><span>CHEM ATLAS<small
05 S4:0005+ >化合物研究工作台</small></span></a><div class="nav-label">研究空间 / WORKSPACE</div><nav id="nav"><
06 S4:0005+ /nav><div class="sidebar-foot"><span class="dot"></span> 本地数据工作空间<small>化合物数据与可解释筛选
07 S4:0005+ 系统<br>V1.0 · 离线计算</small></div></aside>
08 S4:0006  <div class="workspace"><header><span>研究工作台 <b>/</b> <span id="crumb">数据总览</span></span><sp
09 S4:0006+ an class="header-tag">RDKit 化学计算 <span class="dot"></span></span></header><main id="main" ari
10 S4:0006+ a-live="polite"></main><footer>CHEM ATLAS · V1.0 <span>来源标签 ≠ 模型预测 · 原始数据只读保留</span></
11 S4:0006+ footer></div>
12 S4:0007  <div id="toast" role="status"></div>
13 S4:0008  </body></html>
14 S5:0001  :root{--ink:#19313c;--muted:#70818c;--line:#e1e8ec;--bg:#f4f7f9;--teal:#087e8b;--blue:#225bba;--
15 S5:0001+ nav:#112e3b;--pale:#e7f5f3}*{box-sizing:border-box}body{margin:0;font:14px/1.6 "Noto Sans CJK SC
16 S5:0001+ ","Microsoft YaHei",sans-serif;color:var(--ink);background:var(--bg)}aside{position:fixed;inset:
17 S5:0001+ 0 auto 0 0;width:216px;background:var(--nav);color:#d6e5eb;padding:30px 19px;display:flex;flex-d
18 S5:0001+ irection:column}.brand{display:flex;gap:10px;color:white;text-decoration:none;font-size:17px;fon
19 S5:0001+ t-weight:700;letter-spacing:1px}.brand-icon{font-size:39px;color:#7fe2d6;line-height:1.3}.brand
20 S5:0001+ small{display:block;font-size:11px;letter-spacing:2px;color:#a5bfc9;font-weight:400}.nav-label{m
21 S5:0001+ argin:43px 11px 14px;font-size:10px;letter-spacing:1.5px;color:#8ba5af}nav a{display:flex;align-
22 S5:0001+ items:center;gap:13px;color:#b5cbd5;text-decoration:none;padding:12px 15px;margin:5px 0;border-r
23 S5:0001+ adius:7px;font-size:14px}nav a:hover,nav a.active{background:#245261;color:white}nav a.active{bo
24 S5:0001+ x-shadow:inset 3px 0 #5de0cc}nav i{font-style:normal;font-size:19px;width:20px;text-align:center
25 S5:0001+ }.sidebar-foot{margin-top:auto;font-size:12px;padding:18px 10px 0;border-top:1px solid #365361}.
26 S5:0001+ sidebar-foot small{display:block;color:#91adb8;font-size:10px;margin-top:10px;line-height:1.9}.d
27 S5:0001+ ot{display:inline-block;width:7px;height:7px;border-radius:50%;background:#37bd9c;margin-right:6
28 S5:0001+ px}.workspace{margin-left:216px}header{height:66px;padding:0 30px;border-bottom:1px solid var(--
29 S5:0001+ line);background:white;display:flex;align-items:center;justify-content:space-between;color:var(-
30 S5:0001+ -muted);font-size:12px}header b{margin:0 15px;color:#b2c0c7}.header-tag{padding:5px 12px;border:
31 S5:0001+ 1px solid var(--line);border-radius:30px;color:#3c646d}main{padding:28px 30px 16px;min-height:69
32 S5:0001+ 0px}footer{display:flex;justify-content:space-between;padding:14px 30px;color:#94a4ac;font-size:
33 S5:0001+ 10px}.page-head{display:flex;align-items:center;justify-content:space-between;gap:15px;margin-bo
34 S5:0001+ ttom:22px}.eyebrow{color:var(--teal);font-size:10px;letter-spacing:2px;font-weight:700;margin-bo
35 S5:0001+ ttom:4px}h1{font-size:25px;line-height:1.4;margin:0 0 6px;font-weight:700;letter-spacing:.5px}h2
36 S5:0001+ {font-size:16px;margin:0 0 14px}h3{font-size:14px;margin:0 0 8px}p{margin:5px 0 12px}.subtle{col
37 S5:0001+ or:var(--muted);font-size:12px}.metrics{display:grid;grid-template-columns:repeat(4,1fr);gap:15p
38 S5:0001+ x;margin-bottom:20px}.metric{background:white;border:1px solid var(--line);border-radius:10px;pa
39 S5:0001+ dding:18px 20px}.metric strong{display:block;font-size:29px;letter-spacing:-1px;margin:3px 0}.me
40 S5:0001+ tric small{color:var(--muted);font-size:11px}.metric .label{color:#617782;font-size:12px}.metric
41 S5:0001+ :first-child{border-top:3px solid var(--teal);padding-top:16px}.grid2{display:grid;grid-template
42 S5:0001+ -columns:1.1fr 1fr;gap:18px}.panel{background:white;border:1px solid var(--line);border-radius:1
43 S5:0001+ 0px;padding:20px;margin-bottom:18px;overflow:hidden}.panel-title{display:flex;align-items:center
44 S5:0001+ ;justify-content:space-between;margin-bottom:14px}.panel-title h2{margin:0}.badge{display:inline
45 S5:0001+ -block;border-radius:4px;padding:3px 8px;font-size:10px;background:#eaf0f9;color:#4168a0;white-s
46 S5:0001+ pace:nowrap}.badge.green{color:#117b69;background:#e5f4ee}.badge.gray{color:#6f7e89;background:#
47 S5:0001+ eef2f5}.badge.amber{color:#957322;background:#fff5da}.bar-row{margin:15px 0}.bar-label{display:f
48 S5:0001+ lex;justify-content:space-between;font-size:12px;margin-bottom:7px}.bar-track{height:9px;border-
49 S5:0001+ radius:5px;background:#eef2f5;overflow:hidden}.bar{height:100%;border-radius:5px;background:var(
50 S5:0001+ --teal)}.bar.blue{background:#5d84bc}.bar.light{background:#8eb9c5}.flow{display:flex;align-item
51 S5:0001+ s:center;gap:9px;margin:15px 0}.flow span{background:#f2f6f8;padding:8px;border-radius:5px;font-
```

左栏为页内行号及原文件行号；“+”表示同一源码行的续排。源文件全部纳入，空白行不显示。

本页 51 排版行；源码量按原文件统计，折行不增加源程序量。

本页文件：S5 web/style.css

```
01 S5:0001+ size:12px}.flow b{color:#9bb0bb}.feature{padding:12px 0;border-bottom:1px solid #eef2f5}.feature
02 S5:0001+ :last-child{border:0}.feature strong{display:block;font-size:13px}.feature small{font-size:11px;
03 S5:0001+ color:var(--muted)}button,.button{border:0;border-radius:6px;background:var(--teal);color:white;
04 S5:0001+ padding:9px 15px;font:inherit;font-size:12px;cursor:pointer;text-decoration:none;display:inline-
05 S5:0001+ block;white-space:nowrap}button:hover,.button:hover{filter:brightness(.94)}button:disabled{opaci
06 S5:0001+ ty:.4;cursor:not-allowed}.secondary{background:white;color:#3c6572;border:1px solid #d6e1e6}.lin
07 S5:0001+ k{background:none;color:var(--teal);padding:2px 6px}.danger{color:#b14f4f}.actions{display:flex;
08 S5:0001+ gap:8px;align-items:center;flex-wrap:wrap}.form-row{display:flex;gap:12px;align-items:end;flex-w
09 S5:0001+ rap:wrap}.form-row label{flex:1;min-width:115px}.form-row label.wide{flex:3}label{font-size:11px
10 S5:0001+ ;color:#607783;display:block}input,select,textarea{display:block;width:100%;padding:9px 10px;mar
11 S5:0001+ gin-top:6px;border:1px solid #d8e2e7;border-radius:5px;background:#fff;font:inherit;font-size:12
12 S5:0001+ px;color:var(--ink)}input:focus,select:focus,textarea:focus{outline:2px solid #9bd8d2;outline-of
13 S5:0001+ fset:1px}textarea{resize:vertical;min-height:72px}table{width:100%;border-collapse:collapse;font
14 S5:0001+ -size:12px}th{text-align:left;padding:11px 9px;color:#6b828d;font-weight:500;background:#f6f8fa;
15 S5:0001+ border-bottom:1px solid var(--line);font-size:11px}td{padding:11px 9px;border-bottom:1px solid #
16 S5:0001+ edf1f4;vertical-align:middle}tr:last-child td{border-bottom:0}.mono{font-family:ui-monospace,Con
17 S5:0001+ solas,monospace;font-size:11px;word-break:break-all}.smiles{max-width:235px;white-space:nowrap;o
18 S5:0001+ verflow:hidden;text-overflow:ellipsis;color:#7b8f98;font-size:10px;margin-top:2px}.table-wrap{ov
19 S5:0001+ erflow:auto}.table-panel{padding:0}.table-panel .panel-title{padding:16px 18px;margin:0}.pager{d
20 S5:0001+ isplay:flex;justify-content:space-between;align-items:center;padding:12px 18px;border-top:1px so
21 S5:0001+ lid var(--line);color:var(--muted);font-size:11px}.empty{padding:50px 20px;text-align:center;col
22 S5:0001+ or:var(--muted)}.notice{border-left:3px solid #4a9caa;background:#eaf4f6;padding:11px 14px;font-
23 S5:0001+ size:12px;margin-bottom:18px;line-height:1.8}.notice.warning{background:#fff7e9;border-color:#d5
24 S5:0001+ a44b}.error{background:#fff0ee;color:#a74035;border:1px solid #f4d3cd;padding:14px;border-radius
25 S5:0001+ :6px;margin-bottom:16px}.structure{width:100%;height:230px;object-fit:contain;background:white}.
26 S5:0001+ detail-grid{display:grid;grid-template-columns:1fr 1.3fr;gap:18px}.props{display:grid;grid-templ
27 S5:0001+ ate-columns:repeat(5,1fr);gap:8px;margin:14px 0}.prop{background:#f2f7f8;border-radius:6px;paddi
28 S5:0001+ ng:9px;text-align:center}.prop strong{display:block;font-size:16px}.prop span{font-size:10px;col
29 S5:0001+ or:#778b96}.kv{display:grid;grid-template-columns:105px 1fr;gap:8px;font-size:12px;padding:8px 0
30 S5:0001+ ;border-bottom:1px solid #edf1f4}.kv span:first-child{color:#81939d}.payload{font-size:12px}.pay
31 S5:0001+ load details{border-bottom:1px solid var(--line);padding:10px 0}.payload summary{cursor:pointer;
32 S5:0001+ font-weight:600}.payload p{white-space:pre-wrap;overflow-wrap:anywhere;color:#58707b;margin-top:
33 S5:0001+ 8px}.reason{font-size:10px;color:#418273;padding-top:4px}.result-row{display:grid;grid-template-
34 S5:0001+ columns:100px 1fr 100px;gap:16px;align-items:center;border-bottom:1px solid var(--line);padding:
35 S5:0001+ 11px 0}.result-row:last-child{border-bottom:0}.result-row img{width:100px;height:68px;object-fit
36 S5:0001+ :contain}.score{font-size:24px;font-weight:700;color:var(--teal)}.score small{font-size:9px;disp
37 S5:0001+ lay:block;font-weight:400;color:var(--muted)}.compare-grid{display:grid;grid-template-columns:re
38 S5:0001+ peat(3,1fr);gap:15px}.compare-card{border:1px solid var(--line);border-radius:8px;padding:16px}.
39 S5:0001+ compare-card img{height:160px;width:100%;object-fit:contain}.hash{font-size:9px;word-break:break
40 S5:0001+ -all;color:#6f8590}.source-card{padding:14px;border:1px solid var(--line);border-radius:7px;marg
41 S5:0001+ in-bottom:12px}.source-card:last-child{margin-bottom:0}.source-card h3{font-size:12px}#toast{pos
42 S5:0001+ ition:fixed;bottom:24px;right:28px;max-width:440px;background:#173e4b;color:white;border-radius:
43 S5:0001+ 8px;padding:13px 20px;box-shadow:0 6px 25px #102b3633;z-index:50;display:none}.busy{padding:70px
44 S5:0001+ ;text-align:center;color:#6b818c}.split-line{height:1px;background:var(--line);margin:14px 0}.ru
45 S5:0001+ n-summary{background:#f6f9fa;padding:12px;border-radius:5px;font-size:12px;line-height:1.9}.stat
46 S5:0001+ us-row{display:flex;gap:12px;font-size:12px;color:#64808b;flex-wrap:wrap}.small-input{max-width:
47 S5:0001+ 300px}.compare-table td:first-child{color:#728691}.checklist{padding-left:19px;color:#607985;fon
48 S5:0001+ t-size:12px;line-height:2}a{color:var(--teal)}.capture-hide{display:none}@media(max-width:1000px
49 S5:0001+ ){aside{width:180px;padding:24px 12px}.workspace{margin-left:180px}main{padding:20px}.metrics{ga
50 S5:0001+ p:9px}.metric{padding:12px}.metric strong{font-size:23px}.grid2,.detail-grid{grid-template-colum
51 S5:0001+ ns:1fr}.compare-grid{grid-template-columns:1fr}}@media(max-width:650px){aside{position:static;wi
```

本页文件：S5 web/style.css；S6 web/app.js

```
S5:0001+ dth:100%;padding:16px}.nav-label,.sidebar-foot{display:none}nav{display:flex;flex-wrap:wrap}nav
S5:0001+ a{padding:5px;font-size:12px}.workspace{margin:0}header{padding:0 15px}.metrics{grid-template-co
S5:0001+ lumns:1fr 1fr}.page-head{align-items:start}.props{grid-template-columns:repeat(3,1fr)}main{paddi
S5:0001+ ng:15px}footer{padding:15px}.header-tag{display:none}}
S6:0001  'use strict';
S6:0002  const $ = (q) => document.querySelector(q);
S6:0003  const esc = (v) => String(v ?? '').replace(/[&<>"']/g, c => ({'&':'&amp;','<':'&lt;','>':'&gt;',
S6:0003+ '"':'&quot;',"'":'&#39;'}[c]));
S6:0004  const fmt = (n) => Number(n).toLocaleString('zh-CN');
S6:0005  const sourceNames = {compound:'化合物数据集',bbb:'血脑屏障数据集',drugbank:'DrugBank 资料'};
S6:0006  const navItems = [['overview','◈','数据总览'],['explore','⌕','多源检索'],['similar','⌬','相似结构探
S6:0006+ 索'],['screen','⚗','可解释筛选'],['compare','▥','候选分子对比'],['quality','◇','质量与溯源'],['history
S6:0006+ ','◷','筛选记录']];
S6:0007  let candidates;
S6:0008  try {candidates = JSON.parse(localStorage.getItem('chem-candidates') || '[]'); if(!Array.isArray
S6:0008+ (candidates)) candidates=[];} catch {candidates=[];}
S6:0009  let screenResult = null;
S6:0010  let searchParams = {source:'compound',q:'',label:'',page:1,size:8};
S6:0011  let similarityInput = {smiles:'N1C2=C(C3=CC=CC=C3C1)C=CC=C2c9ncc[nH]9',threshold:.4};
S6:0012  let screenParams = {source:'compound',mw_max:500,logp_max:5,hbd_max:5,hba_max:10,psa_max:140};
S6:0013  let sequence=0;
S6:0014  const api = async (path, params, options) => {
S6:0015    const response = await fetch('/api/'+path+(params?'?'+new URLSearchParams(params):''), options
S6:0015+ );
S6:0016    const body = await response.json();
S6:0017    if(!response.ok) throw new Error(body.error || '请求失败');
S6:0018    return body;
S6:0019  };
S6:0020  function toast(message){$('#toast').textContent=message;$('#toast').style.display='block';setTim
S6:0020+ eout(()=>$('#toast').style.display='none',2800);}
S6:0021  function heading(title, subtitle, action=''){return `<div class="page-head"><div><div class="eye
S6:0021+ brow">CHEM ATLAS / RESEARCH WORKSPACE</div><h1>${esc(title)}</h1><div class="subtle">${esc(subti
S6:0021+ tle)}</div></div><div class="actions">${action}</div></div>`;}
S6:0022  function badge(source){return `<span class="badge ${source==='compound'?'green':source==='bbb'?'
S6:0022+ ':'gray'}">${sourceNames[source]||esc(source)}</span>`;}
S6:0023  function button(text, action, cls='secondary', extra=''){return `<button class="${cls}" data-act
S6:0023+ ion="${action}" ${extra}>${text}</button>`;}
S6:0024  function linkDetail(id){return `#detail/${encodeURIComponent(id)}`;}
S6:0025  function empty(message){return `<div class="empty">${message}</div>`;}
S6:0026  function image(id){return `/api/structure?id=${encodeURIComponent(id)}`;}
S6:0027  function value(v){return v == null ? '—' : esc(v);}
S6:0028  function sourceOptions(selected,all=true){return `${all?'<option value="">全部来源</option>':''}$
S6:0028+ {Object.entries(sourceNames).map(([k,v])=>`<option value="${k}" ${selected===k?'selected':''}>${
S6:0028+ v}</option>`).join('')}`;}
S6:0029  function metric(label,n,note){return `<div class="metric"><div class="label">${label}</div><stro
S6:0029+ ng>${fmt(n)}</strong><small>${note}</small></div>`;}
S6:0030  function table(items, explanations=false){
S6:0031    if(!items.length)return empty('没有符合条件的记录，请调整关键词或筛选条件。');
S6:0032    return `<div class="table-wrap"><table><thead><tr><th>记录 / 分子表示</th><th>数据来源</th><th>MW<
S6:0032+ /th><th>LogP</th><th>TPSA</th><th>来源标签</th><th>操作</th></tr></thead><tbody>${items.map(r=>`<t
S6:0032+ r><td><a href="${linkDetail(r.id)}"><strong>${esc(r.name)}</strong></a><div class="smiles" title
```

本页文件：S6 web/app.js

```
S6:0032+ ="${esc(r.smiles||'该来源不含结构')}">${esc(r.smiles||'该来源不含结构')}</div>${explanations?`<div cl
S6:0032+ ass="reason">${r.reasons.map(esc).join(' · ')}</div>`:''}</td><td>${badge(r.source)}</td><td>${v
S6:0032+ alue(r.mw)}</td><td>${value(r.logp)}</td><td>${value(r.psa)}</td><td><span class="badge gray">${
S6:0032+ esc(r.label||'未提供')}</span></td><td>${button(candidates.includes(r.id)?'已加入':'加入对比','add',
S6:0032+ 'link',`data-id="${esc(r.id)}" ${!r.smiles?'disabled':''}`)}</td></tr>`).join('')}</tbody></tabl
S6:0032+ e></div>`;
S6:0033  }
S6:0034  async function overview(){
S6:0035    const q = await api('overview');
S6:0036    return heading('从数据出发，探索分子空间','整合本地化学数据 · 连接结构、性质与筛选证据',`<a class="button
S6:0036+ " href="#explore">开始检索 →</a>`)+
S6:0037      `<div class="metrics">${metric('全部来源记录',q.sources.reduce((n,s)=>n+s.rows,0),'3 个本地数据
S6:0037+ 文件')}${metric('可解析结构记录',q.structures,'RDKit 结构标准化')}${metric('唯一分子结构',q.unique_stru
S6:0037+ ctures,'基于规范化 SMILES')}${metric('血脑屏障来源标签',10648,'实验 / 文献标签，非模型预测')}</div>`+
S6:0038      `<div class="grid2"><section class="panel"><div class="panel-title"><h2>数据资产分布</h2><span
S6:0038+  class="badge green">本地索引已就绪</span></div>${q.sources.map((s,i)=>`<div class="bar-row"><div
S6:0038+ class="bar-label"><span>${sourceNames[s.source]}</span><strong>${fmt(s.rows)} 条</strong></div><
S6:0038+ div class="bar-track"><div class="bar ${i===1?'blue':i===2?'light':''}" style="width:${s.rows/14
S6:0038+ 313*100}%"></div></div><small class="subtle">${esc(s.path)}</small></div>`).join('')}<div class=
S6:0038+ "split-line"></div><div class="subtle">数据来源分别保留；仅对完全相同的规范化结构展示关联。</div></sectio
S6:0038+ n>
S6:0039     <section class="panel"><h2>可复核的研究流程</h2><div class="flow"><span>检索数据</span><b>→</b><
S6:0039+ span>探索结构</span><b>→</b><span>筛选归档</span></div><div class="feature"><strong>01 · 结构相似度有
S6:0039+ 依据</strong><small>Morgan 指纹与 Tanimoto 系数，展示算法及阈值。</small></div><div class="feature"><s
S6:0039+ trong>02 · 每个筛选结果可解释</strong><small>逐条展示性质值与条件，支持候选分子横向对比。</small></div><div
S6:0039+  class="feature"><strong>03 · 筛选记录带数据指纹</strong><small>保存命中 ID、条件、来源 SHA256 与计算库
S6:0039+ 版本。</small></div><a class="button secondary" href="#screen">进入可解释筛选</a></section></div><di
S6:0039+ v class="notice">性质由 RDKit ${esc(q.rdkit)} 重新计算，原始字段保存在详情中。DrugBank 资料没有 SMILES，
S6:0039+ 不参与结构计算。</div>`;
S6:0040  }
S6:0041  async function explore(){
S6:0042    const r=await api('search',searchParams);
S6:0043    return heading('多源检索','按来源、关键词和来源标签定位研究数据',`<a class="button secondary" href="/a
S6:0043+ pi/export?${new URLSearchParams(searchParams)}">导出全部匹配 CSV</a>`)+
S6:0044      `<form id="search-form" class="panel form-row"><label>数据来源<select name="source">${sourceO
S6:0044+ ptions(searchParams.source)}</select></label><label class="wide">关键词 / ID / SMILES / 原文<input
S6:0044+  name="q" placeholder="例如 tmy0、DB00001 或 thrombin" value="${esc(searchParams.q)}"></label><la
S6:0044+ bel>来源标签<select name="label">${[['','全部标签'],['active','active'],['inactive','inactive'],['0
S6:0044+ ','BBB 0'],['1','BBB 1']].map(([k,v])=>`<option value="${k}" ${searchParams.label===k?'selected'
S6:0044+ :''}>${v}</option>`).join('')}</select></label><button type="submit">查询</button></form>`+
S6:0045      `<section class="panel table-panel"><div class="panel-title"><h2>查询结果 <span class="subtle
S6:0045+ ">${fmt(r.total)} 条记录</span></h2><span class="subtle">MW: g/mol · TPSA: Å²</span></div>${table
S6:0045+ (r.items)}<div class="pager"><span>第 ${r.page} / ${Math.max(1,Math.ceil(r.total/r.size))} 页 ·
S6:0045+ 每页 ${r.size} 条</span><div class="actions">${button('上一页','prev','secondary',r.page===1?'disa
S6:0045+ bled':'')}${button('下一页','next','secondary',r.page*r.size>=r.total?'disabled':'')}</div></div>
S6:0045+ </section>`;
S6:0046  }
S6:0047  async function detail(id){
S6:0048    const r = await api('detail',{id});
S6:0049    const fields = Object.entries(r.payload);
S6:0050    return heading(r.name+' · 记录详情','原始字段、计算性质与来源定位',`<a class="button secondary" href
```

本页文件：S6 web/app.js

```
S6:0050+ ="#explore">返回检索</a>${r.smiles?button('加入候选对比','add','',`data-id="${esc(r.id)}"`):''}`)+
S6:0051    `<div class="detail-grid"><section class="panel"><div class="panel-title"><h2>分子结构</h2>${ba
S6:0051+ dge(r.source)}</div>${r.canonical?`<img class="structure" src="${image(id)}" alt="${esc(r.name)}
S6:0051+  二维分子结构"><div class="mono">${esc(r.canonical)}</div>`:empty('此记录不含可计算结构')}<div class="
S6:0051+ split-line"></div><div class="kv"><span>来源文件</span><span>${esc({compound:'compound_info.csv',
S6:0051+ bbb:'Dataset/BBB_10648.csv',drugbank:'DrugBankAll.xlsx'}[r.source])}</span></div><div class="kv"
S6:0051+ ><span>原文件行号</span><span>第 ${r.source_row} 行（含表头）</span></div><div class="kv"><span>来源
S6:0051+ 标签</span><span>${esc(r.label||'未提供')}</span></div>${r.smiles?button('以此结构查相似分子','use-si
S6:0051+ milar','secondary',`data-smiles="${esc(r.smiles)}"`):''}</section>
S6:0052    <section class="panel"><h2>${r.canonical?'计算性质与来源证据':'药物资料原文'}</h2>${r.canonical?`<d
S6:0052+ iv class="props">${['mw','logp','hbd','hba','psa'].map(p=>`<div class="prop"><span>${p.toUpperCa
S6:0052+ se()}</span><strong>${value(r[p])}</strong></div>`).join('')}</div><p class="subtle">MW: g/mol；
S6:0052+ TPSA: Å²；HBD/HBA: 氢键供体/受体数。所有值由本版本 RDKit 统一计算。</p>`:'<div class="notice">该来源未提
S6:0052+ 供分子结构，不显示推断性质。点击字段名称展开原始内容。</div>'}<div class="payload">${fields.map(([k,v],i)
S6:0052+ =>`<details ${i<2?'open':''}><summary>${esc(k)}</summary><p>${esc(v||'原始数据未提供')}</p></detai
S6:0052+ ls>`).join('')}</div></section></div>
S6:0053    <section class="panel"><h2>相同规范化结构的来源记录 <span class="subtle">${r.related_total} 条</spa
S6:0053+ n></h2>${r.related.length?`<div class="actions">${r.related.map(x=>`<a class="button secondary"
S6:0053+ href="${linkDetail(x.id)}">${esc(x.name)} · ${esc(x.label||'无标签')}</a>`).join('')}</div><p cla
S6:0053+ ss="subtle">最多展示 20 条；结构完全相同的来源记录不代表实验条件相同。</p>`:'<p class="subtle">未发现其他相
S6:0053+ 同规范化结构记录，或此记录没有结构。</p>'}</section>`;
S6:0054  }
S6:0055  async function similar(){
S6:0056   return heading('相似结构探索','基于结构指纹寻找相似分子，保留各来源记录')+
S6:0057   `<section class="panel"><form id="similar-form"><label>查询分子 SMILES<textarea name="smiles" re
S6:0057+ quired>${esc(similarityInput.smiles)}</textarea></label><div class="form-row" style="margin-top:
S6:0057+ 14px"><label style="max-width:200px">最低 Tanimoto 相似度<input name="threshold" type="number" min
S6:0057+ ="0" max="1" step="0.05" value="${similarityInput.threshold}" required></label><button type="sub
S6:0057+ mit">计算相似分子</button><span class="subtle">Morgan · radius 2 · 2048 bits</span></div></form></
S6:0057+ section><div class="notice">相似度表示指纹重合程度，不等同于生物活性、血脑屏障通透性或药效。结果最多展示前 60
S6:0057+  条。</div><div id="similar-results">${empty('输入 SMILES 或从记录详情带入结构，开始探索。')}</div>`;
S6:0058  }
S6:0059  async function screen(){
S6:0060   return heading('可解释筛选','用透明的性质条件缩小候选范围，每条命中均附有判断依据',button('恢复默认条件','re
S6:0060+ set-screen'))+
S6:0061   `<section class="panel"><form id="screen-form"><div class="form-row"><label>候选来源<select name
S6:0061+ ="source"><option value="compound" ${screenParams.source==='compound'?'selected':''}>化合物数据集<
S6:0061+ /option><option value="bbb" ${screenParams.source==='bbb'?'selected':''}>血脑屏障数据集</option><o
S6:0061+ ption value="all" ${screenParams.source==='all'?'selected':''}>全部结构记录</option></select></lab
S6:0061+ el>${[['mw_max','MW 上限 (g/mol)'],['logp_max','LogP 上限'],['hbd_max','HBD 上限'],['hba_max','HBA
S6:0061+  上限'],['psa_max','TPSA 上限 (Å²)']].map(([key,label])=>`<label>${label}<input name="${key}" typ
S6:0061+ e="number" step="any" min="${key==='logp_max'?-20:0}" max="${key==='logp_max'?20:10000}" value="
S6:0061+ ${screenParams[key]}" required></label>`).join('')}<button type="submit">执行筛选</button></div><
S6:0061+ /form></section><div class="notice">所有条件同时满足，边界值计入结果。默认条件是研究筛选参考，不构成成药性、
S6:0061+ 有效性或安全性结论。</div><div id="screen-results">${screenResult?screenOutput():empty('设置上限并执行
S6:0061+ 筛选，将显示命中数及逐项解释。')}</div>`;
S6:0062  }
S6:0063  function screenOutput(){
S6:0064   const r=screenResult;
S6:0065   return `<section class="panel table-panel"><div class="panel-title"><h2>命中 ${fmt(r.total)} 条
S6:0065+ <span class="badge green">5 项性质条件同时满足</span></h2><a class="button secondary" href="/api/ex
```

本页文件：S6 web/app.js

```
S6:0065+ port?kind=screen&${new URLSearchParams(r.params)}">导出全部命中 CSV</a></div>${table(r.items.slice
S6:0065+ (0,6),true)}<div class="pager"><span>预览前 ${Math.min(r.total,6)} 条 · 按 MW 升序 · 导出包含全部 ${f
S6:0065+ mt(r.total)} 条</span></div></section><form id="save-form" class="panel form-row"><label>筛选记录
S6:0065+ 名称<input name="name" maxlength="100" placeholder="例如：低分子量候选初筛" required></label><button
S6:0065+ type="submit">保存筛选记录</button><span class="subtle">保存条件、全部命中 ID 和数据来源指纹</span></for
S6:0065+ m>`;
S6:0066  }
S6:0067  async function compare(){
S6:0068   const records=await Promise.all(candidates.map(id=>api('detail',{id})));
S6:0069   return heading('候选分子对比','并列检查结构和理化性质，候选列表保存在当前浏览器',button('清空候选','clear-c
S6:0069+ ompare'))+
S6:0070   (!records.length?`<section class="panel">${empty('还没有候选分子。到检索或筛选页面点击“加入对比”，最多可
S6:0070+ 加入 3 条。')}<a class="button" href="#explore">去检索分子</a></section>`:
S6:0071   `<section class="panel"><div class="compare-grid">${records.map(r=>`<div class="compare-card"><
S6:0071+ div class="panel-title"><strong>${esc(r.name)}</strong>${button('移除','remove','link',`data-id="
S6:0071+ ${esc(r.id)}"`)}</div>${badge(r.source)}<img src="${image(r.id)}" alt="${esc(r.name)} 结构"><div
S6:0071+ class="mono">${esc(r.smiles)}</div></div>`).join('')}</div><div class="split-line"></div><table
S6:0071+ class="compare-table"><thead><tr><th>对比指标</th>${records.map(r=>`<th>${esc(r.name)}</th>`).joi
S6:0071+ n('')}</tr></thead><tbody>${[['mw','分子量 MW (g/mol)'],['logp','脂水分配 LogP'],['hbd','氢键供体 HB
S6:0071+ D'],['hba','氢键受体 HBA'],['psa','极性表面积 TPSA (Å²)'],['label','原始来源标签']].map(([k,label])=>
S6:0071+ `<tr><td>${label}</td>${records.map(r=>`<td><strong>${value(r[k])}</strong></td>`).join('')}</tr
S6:0071+ >`).join('')}</tbody></table></section><div class="notice">比较使用同一版本 RDKit 计算的性质。来源标签
S6:0071+ 来自各原始文件，不能作为跨实验条件的效力排序。</div>`);
S6:0072  }
S6:0073  async function quality(){
S6:0074   const q=await api('quality');
S6:0075   return heading('质量与溯源','从记录计数到文件指纹，核对研究结果所依据的数据')+
S6:0076   `<div class="metrics">${metric('来源文件',3,'SHA256 内容校验')}${metric('非法结构',q.sources.reduce
S6:0076+ ((n,s)=>n+s.invalid,0),'仅统计有结构字段的来源')}${metric('缺失结构',q.sources.reduce((n,s)=>n+s.miss
S6:0076+ ing,0),'DrugBank 无结构为来源设计')}${metric('源内重复记录',q.sources.reduce((n,s)=>n+s.duplicates,0)
S6:0076+ ,'保留重复行，呈现来源差异')}</div>`+
S6:0077   `<div class="grid2"><section class="panel"><h2>文件指纹与导入证据</h2>${q.sources.map(s=>`<div cla
S6:0077+ ss="source-card"><h3>${sourceNames[s.source]} <span class="subtle">${fmt(s.rows)} 条</span></h3>
S6:0077+ <div class="subtle">${esc(s.path)} · ${new Date(s.imported).toLocaleDateString('zh-CN')}</div><d
S6:0077+ iv class="hash">SHA256 ${s.sha256}</div><div class="status-row"><span>非法 ${s.invalid}</span><sp
S6:0077+ an>缺失 ${s.missing}</span><span>源内重复 ${s.duplicates}</span></div></div>`).join('')}</section>
S6:0077+ <section class="panel"><h2>质量规则与解释边界</h2><div class="feature"><strong>结构标准化</strong><sm
S6:0077+ all>RDKit ${q.rdkit} · 规范化 SMILES · 保留来源原文</small></div><div class="feature"><strong>重复记
S6:0077+ 录识别</strong><small>${esc(q.quality_note)}</small></div><div class="feature"><strong>来源字段与计
S6:0077+ 算字段并存</strong><small>主表显示统一计算值；记录详情中可展开原始字段进行核对。</small></div><div class="f
S6:0077+ eature"><strong>关联证据</strong><small>仅展示规范化 SMILES 完全一致的记录，不按药物描述猜测关联。</small>
S6:0077+ </div><div class="feature"><strong>研究记录复现</strong><small>筛选归档保留参数、全部命中 ID、前 100 条
S6:0077+ 结果快照、文件指纹及计算库版本。</small></div></section></div>`;
S6:0078  }
S6:0079  async function history(){
S6:0080   const rows=await api('runs');
S6:0081   return heading('筛选记录','查看已归档的筛选条件、结果标识和来源证据',`<a class="button" href="#screen">
S6:0081+ 创建筛选记录</a>`)+
S6:0082   `<section class="panel table-panel"><div class="panel-title"><h2>已保存 ${rows.length} 条记录</h2
S6:0082+ ><span class="subtle">持久保存于本地 SQLite</span></div>${rows.length?`<table><thead><tr><th>记录名
S6:0082+ 称</th><th>创建时间</th><th>命中数量</th><th>操作</th></tr></thead><tbody>${rows.map(r=>`<tr><td><a
```

本页文件：S6 web/app.js

```
S6:0082+ href="#run/${r.id}"><strong>${esc(r.name)}</strong></a><div class="subtle">筛选编号 #${r.id}</div
S6:0082+ ></td><td>${new Date(r.created).toLocaleString('zh-CN')}</td><td>${fmt(r.count)}</td><td><a href
S6:0082+ ="/api/export?kind=run&id=${r.id}" class="button secondary">导出 JSON</a></td></tr>`).join('')}</
S6:0082+ tbody></table>`:empty('暂无筛选记录。执行筛选后输入名称并保存。')}</section>`;
S6:0083  }
S6:0084  async function runDetail(id){
S6:0085   const r=await api('run',{id});
S6:0086   return heading(r.name,'筛选归档 #'+r.id+' · '+new Date(r.created).toLocaleString('zh-CN'),`<a cl
S6:0086+ ass="button secondary" href="#history">返回记录</a><a class="button" href="/api/export?kind=run&i
S6:0086+ d=${r.id}">下载证据 JSON</a>`)+
S6:0087   `<div class="grid2"><section class="panel"><h2>筛选条件快照</h2>${Object.entries(r.params).map(([
S6:0087+ k,v])=>`<div class="kv"><span>${esc(k)}</span><strong>${esc(v)}</strong></div>`).join('')}<div c
S6:0087+ lass="run-summary">命中 <strong>${fmt(r.ids.length)}</strong> 条来源记录<br>计算库 RDKit ${esc(r.pro
S6:0087+ venance.rdkit)}<br>数值条件全部为包含边界的上限判断。</div></section><section class="panel"><h2>数据版
S6:0087+ 本证据</h2>${r.provenance.sources.map(s=>`<div class="source-card"><h3>${esc(s.path)}</h3><div cl
S6:0087+ ass="hash">${esc(s.sha256)}</div></div>`).join('')}<div class="notice">JSON 包含全部命中 ID 和前 10
S6:0087+ 0 条结果快照。复现时需同时核对原文件校验值、计算库版本与筛选参数。</div></section></div><section class="pan
S6:0087+ el"><h2>命中标识预览 <span class="subtle">前 24 条 / 全部 ${fmt(r.ids.length)} 条</span></h2><div cl
S6:0087+ ass="actions">${r.ids.slice(0,24).map(id=>`<a class="badge green" href="${linkDetail(id)}">${esc
S6:0087+ (id)}</a>`).join('')}</div></section>`;
S6:0088  }
S6:0089  async function render(){
S6:0090   const current=++sequence;
S6:0091   const [page, id] = (location.hash.slice(1)||'overview').split('/');
S6:0092   $('#nav').innerHTML=navItems.map(([key,icon,label])=>`<a href="#${key}" class="${key===page?'ac
S6:0092+ tive':''}"><i>${icon}</i>${label}${key==='compare'&&candidates.length?` <small>(${candidates.len
S6:0092+ gth})</small>`:''}</a>`).join('');
S6:0093   $('#crumb').textContent=(navItems.find(x=>x[0]===page)||['','','记录详情'])[2];
S6:0094   $('#main').innerHTML='<div class="busy">正在读取本地研究数据…</div>';
S6:0095   try {
S6:0096    const handlers={overview,explore,similar,screen,compare,quality,history};
S6:0097    const html=page==='detail'?await detail(decodeURIComponent(id)):page==='run'?await runDetail(i
S6:0097+ d):await (handlers[page]||overview)();
S6:0098    if(current!==sequence)return;
S6:0099    $('#main').innerHTML=html;
S6:0100    window.scrollTo(0,0);
S6:0101   }catch(e){if(current===sequence)$('#main').innerHTML=`<div class="error">${esc(e.message)}</div
S6:0101+ >`;}
S6:0102  }
S6:0103  document.addEventListener('click',async e=>{
S6:0104   const b=e.target.closest('[data-action]');if(!b)return;
S6:0105   try {
S6:0106    const action=b.dataset.action;
S6:0107    if(action==='add'){
S6:0108     if(candidates.includes(b.dataset.id)){toast('该分子已在候选列表中');return;}
S6:0109     if(candidates.length>=3){toast('最多对比 3 个分子，请先到候选分子对比页移除条目');return;}
S6:0110     candidates.push(b.dataset.id);localStorage.setItem('chem-candidates',JSON.stringify(candidate
S6:0110+ s));b.textContent='已加入';toast('已加入候选分子对比');
S6:0111    }else if(action==='remove'||action==='clear-compare'){
S6:0112     candidates=action==='clear-compare'?[]:candidates.filter(id=>id!==b.dataset.id);localStorage.
S6:0112+ setItem('chem-candidates',JSON.stringify(candidates));await render();
```

本页文件：S6 web/app.js

```
S6:0113     }else if(action==='prev'||action==='next'){
S6:0114      searchParams.page += action==='next'?1:-1;await render();
S6:0115     }else if(action==='use-similar'){
S6:0116      similarityInput.smiles=b.dataset.smiles;location.hash='similar';
S6:0117     }else if(action==='reset-screen'){
S6:0118      screenParams={source:'compound',mw_max:500,logp_max:5,hbd_max:5,hba_max:10,psa_max:140};scree
S6:0118+ nResult=null;await render();
S6:0119     }
S6:0120    }catch(err){toast(err.message);}
S6:0121   });
S6:0122   document.addEventListener('submit',async e=>{
S6:0123    const form=e.target;if(!['search-form','similar-form','screen-form','save-form'].includes(form.
S6:0123+ id))return;e.preventDefault();
S6:0124    const params=Object.fromEntries(new FormData(form));
S6:0125    const submit=form.querySelector('button[type="submit"]');if(submit)submit.disabled=true;
S6:0126    const requestSequence=sequence;
S6:0127    try{
S6:0128     if(form.id==='search-form'){searchParams={...searchParams,...params,page:1};await render();}
S6:0129     if(form.id==='similar-form'){
S6:0130      similarityInput=params;$('#similar-results').innerHTML='<div class="busy">正在计算结构指纹相似度…
S6:0130+ </div>';
S6:0131      const r=await api('similar',params);if(requestSequence!==sequence)return;
S6:0132      $('#similar-results').innerHTML=`<section class="panel"><div class="panel-title"><h2>匹配 ${fm
S6:0132+ t(r.total)} 条 <span class="subtle">当前展示前 ${Math.min(r.total,6)} 条</span></h2><span class="b
S6:0132+ adge green">阈值 ≥ ${r.threshold}</span></div>${r.items.length?r.items.slice(0,6).map(x=>`<div cl
S6:0132+ ass="result-row"><img src="${image(x.id)}" alt="${esc(x.name)} 结构"><div><a href="${linkDetail(x
S6:0132+ .id)}"><strong>${esc(x.name)}</strong></a> ${badge(x.source)}<div class="smiles">${esc(x.smiles)
S6:0132+ }</div><span class="subtle">MW ${x.mw} · LogP ${x.logp}</span> ${button('加入对比','add','link',`
S6:0132+ data-id="${esc(x.id)}"`)}</div><div class="score">${x.similarity.toFixed(3)}<small>TANIMOTO 相似
S6:0132+ 度</small></div></div>`).join(''):empty('该阈值下没有匹配结构，请降低阈值。')}<details><summary class=
S6:0132+ "subtle">展开其余结果（最多前 60 条）</summary>${table(r.items.slice(6))}</details></section>`;
S6:0133     }
S6:0134     if(form.id==='screen-form'){
S6:0135      screenParams=params;const r=await api('screen',params);if(requestSequence!==sequence)return;s
S6:0135+ creenResult=r;$('#screen-results').innerHTML=screenOutput();
S6:0136     }
S6:0137     if(form.id==='save-form'){
S6:0138      const r=await api('runs',null,{method:'POST',headers:{'Content-Type':'application/json'},body
S6:0138+ :JSON.stringify({name:params.name,params:screenResult.params,ids:screenResult.ids})});
S6:0139      if(requestSequence!==sequence)return;toast('筛选记录已保存');location.hash='run/'+r.id;
S6:0140     }
S6:0141    }catch(err){
S6:0142     if(requestSequence!==sequence)return;
S6:0143     const target=form.id==='similar-form'?$('#similar-results'):form.id==='screen-form'?$('#screen
S6:0143+ -results'):null;
S6:0144     if(target)target.innerHTML=`<div class="error">${esc(err.message)}</div>`;else toast(err.messa
S6:0144+ ge);
S6:0145    }finally{if(submit)submit.disabled=false;}
S6:0146   });
S6:0147   window.addEventListener('hashchange',render);
S6:0148   render();
```